1. Данные: от файла до статистики

Программа читает измерения метеостанции из CSV, считает минимум, максимум и среднее для четырёх показателей, затем показывает первые пять записей с переводом единиц. Основная идея работы: один и тот же вызов функции может выбирать разную реализацию в зависимости от типа аргумента.

Как разделён код

weather.hpp описывает типы и объявления функций. Директива **#pragma once** защищает от повторного подключения заголовка. **weather.cpp** содержит чтение, расчёты, преобразования и отчёт; **main.cpp** открывает файл и запускает обработку. Объявления сообщают компилятору, какие функции доступны, а при сборке оба .cpp объединяются в программу.

Структура Measurement: одна запись CSV

time хранится как std::string, **temperature** и **pressure** как double, **humidity** как int, **wind** как float. Это именно типы из задания. Структура объединяет значения одного момента времени. std::vector<Measurement> хранит все записи и растёт по мере чтения, поэтому число 48 нигде не зашито.

readMeasurements(input): разбор строк

Функция принимает поток по ссылке и возвращает вектор измерений. Первый std::getline пропускает заголовок. Цикл читает остальные строки, а std::istringstream позволяет разбирать каждую строку как отдельный поток. Вызовы getline с разделителем «,» извлекают поля в порядке CSV; последнее поле читается до конца строки.

std::stod получает double, **std::stoi** получает int, **std::stof** получает float. После заполнения Measurement вызов push_back добавляет запись в вектор. Формат и значения считаются корректными по условию, поэтому дополнительной проверки CSV нет. Если после заголовка нет строк, возвращается пустой вектор.

WeatherStatistics и calculateStatistics(measurements)

WeatherStatistics хранит по три результата для каждого показателя: минимум, максимум, среднее. Границы сохраняют тип исходной серии: double для температуры и давления, int для влажности, float для ветра. Все четыре средних имеют тип double.

calculateStatistics принимает вектор по const-ссылке: данные не копируются и не меняются. Из первой записи берутся начальные границы; средние начинаются с нуля. В цикле std::min и std::max обновляют границы, а к среднему прибавляется значение / count. Сумма таких долей равна сумме значений, делённой на их число.

count заранее превращается в double через static_cast. Поэтому, например, влажности 50 и 51 дают 50 / 2.0 + 51 / 2.0 = **50.5**, а не 50. Функция требует непустой вектор; это обеспечивает printReport до её вызова. Собственных шаблонов в решении нет; min, max и vector используются из стандартной библиотеки.

2. Типы и выбор перегрузок

Классы Celsius и Fahrenheit

Оба класса хранят одно закрытое поле **value_** типа double. Отдельные типы позволяют различать шкалы по типу аргумента: Celsius(68.0) и Fahrenheit(68.0) обозначают разные температуры, хотя внутри хранится одно и то же число.

Celsius::Celsius(double) записывает число в value_ через список инициализации. **Fahrenheit::Fahrenheit(double)** делает то же для другой шкалы. Оба конструктора объявлены **explicit**: запись Celsius t = 20.0 запрещена, а Celsius t(20.0) разрешена. Это заставляет явно указать шкалу и не позволяет случайному double автоматически стать температурным объектом.

Celsius::value() const и **Fahrenheit::value() const** возвращают хранимое число. const означает, что метод не меняет объект; его можно вызвать и у константной температуры. Значение выдаётся копией, поэтому через результат нельзя изменить закрытое поле.

toCelsius: одно имя, две реализации

toCelsius(Celsius) возвращает переданный объект без изменения: он уже хранит градусы Цельсия. **toCelsius(Fahrenheit)** читает значение через value(), вычисляет **(F - 32.0) * 5.0 / 9.0** и явно создаёт Celsius из результата. Тип возвращаемого значения в обеих версиях - Celsius.

Компилятор выбирает перегрузку по типу входного объекта. Вызов toCelsius(Fahrenheit(68.0)) использует формулу и даёт 20.0 °C. Вызов toCelsius(Celsius(16.1)) сохраняет 16.1 °C. Возвращаемый тип сам по себе не является основанием для перегрузки; здесь различаются параметры.

Остальные преобразования единиц

toFahrenheit(Celsius) возвращает Fahrenheit после расчёта **C * 9.0 / 5.0 + 32.0**. Например, 16.1 °C превращаются в 60.98 °F. Дробные константы сохраняют вещественную арифметику. Исходный объект при этом не меняется.

toKilometersPerHour(float) принимает скорость в м/с и возвращает результат умножения на **3.6**. Константа имеет тип double, поэтому float повышается до double, и результат тоже имеет тип double. Для 4.7 м/с выводится 16.92 км/ч; исходная скорость остаётся float.

printValue: формат зависит от типа числа

Три обычные перегрузки принимают поток output по ссылке и одно число. **printValue(output, int)** выводит целое без дробной части. **printValue(output, float)** включает fixed и setprecision(1): один знак после точки. **printValue(output, double)** включает fixed и setprecision(2): два знака после точки.

Например, влажность 83 печатается как **83**, ветер 4.7f как **4.7**, температура 16.1 как **16.10**. Средняя влажность имеет тип double и выводится как 67.25. Это показывает, почему важно сохранить типы границ: float-граница ветра попадёт в свою перегрузку, int-граница влажности - в свою.

fixed и setprecision меняют настройки потока, а не сами числа. Каждая вещественная перегрузка заново задаёт нужную точность, поэтому предыдущий вывод double не заставит следующий float печататься с двумя знаками. Вывод int не получает дробную часть даже после fixed.

3. Сборка отчёта и запуск программы

printReport(output, measurements): порядок работы

1. Проверка пустого набора. Если вектор пуст, функция пишет «Измерения отсутствуют.» и сразу завершается. Поэтому calculateStatistics не обращается к несуществующей первой записи и не делит на ноль.

2. Статистика. Функция получает WeatherStatistics и выводит четыре строки: температура, влажность, давление и ветер. Каждая граница и каждое среднее передаются в printValue. Так формат выбирается перегрузкой по настоящему типу поля, а не вручную в каждой строке отчёта.

3. Первые записи. shownCount равен минимуму из размера вектора и 5. Цикл берёт записи по индексам от 0 до shownCount - 1. В каждой строке печатаются время, температура в °C и °F, влажность, давление, ветер в м/с и км/ч. Исходные числовые значения также выводятся через printValue. Если записей меньше пяти, выводятся все доступные.

4. Демонстрация обеих перегрузок. Создаётся Fahrenheit(68.0), затем toCelsius даёт 20.0 °C. Отдельно создаётся Celsius из температуры первой записи и передаётся в другую toCelsius: значение сохраняется. На экране числа представлены как 68.00 и 20.00, поскольку это double и для него установлены два знака.

5. Проверенное преобразование в int. Первая температура сравнивается с std::numeric_limits<int>::min() и max(). Только если она внутри диапазона, выполняется static_cast<int>(temperature). Иначе печатается сообщение и преобразование пропускается. По условию исходные числа конечны.

static_cast к int отбрасывает дробную часть **в сторону нуля**: 16.1 даёт 16, а -2.9 даёт -2. Это не округление до ближайшего целого и не округление вниз. Результат используется только для демонстрационной строки; ни исходные измерения, ни статистика не теряют дробную часть.

main(argc, argv): вход в программу

main берёт путь из argv[1], если он передан; иначе использует weather_day.csv в текущей папке. std::ifstream открывает файл. При неудаче сообщение на русском направляется в std::cerr, а return 1 обозначает ошибку. При успехе вызываются readMeasurements и printReport, затем return 0 завершает программу. Поток файла закрывается автоматически при выходе из main.

Как собрать и проверить результат

Из папки lab4: **g++ -std=c++17 -Wall -Wextra -Wpedantic main.cpp weather.cpp -o weather**. Запуск: **./weather**, в Windows: **.\weather.exe**. Другой CSV можно передать первым аргументом. Исходники и сообщения записаны в UTF-8; для терминала Windows можно предварительно выполнить **chcp 65001**.

Для приложенного файла из 48 записей: температура 13.80 / 31.40 / 22.15 °C; влажность 35 / 87 / 67.25 %; давление 987.60 / 1033.20 / 1013.69 гПа; ветер 0.4 / 14.0 / 4.34 м/с. В каждом наборе указаны минимум, максимум и среднее.

Для объяснения на защите достаточно проследить одну запись: строка CSV → Measurement с нужными типами → обновление статистики → выбор printValue по типу → отдельные объекты Celsius/Fahrenheit для преобразований. Печать и демонстрационное приведение к int не изменяют измерения.

Условие: github.com/mvodya/vvsu-study, 2026-S2/cpp-programming/labs/04-overloads-conversions.